

Observability & Monitoring in DevOps

A Technical Deep Dive into Prometheus & Grafana

4th-Year CSE Practical Lecture Series

Gemini Notebook

MONITORING

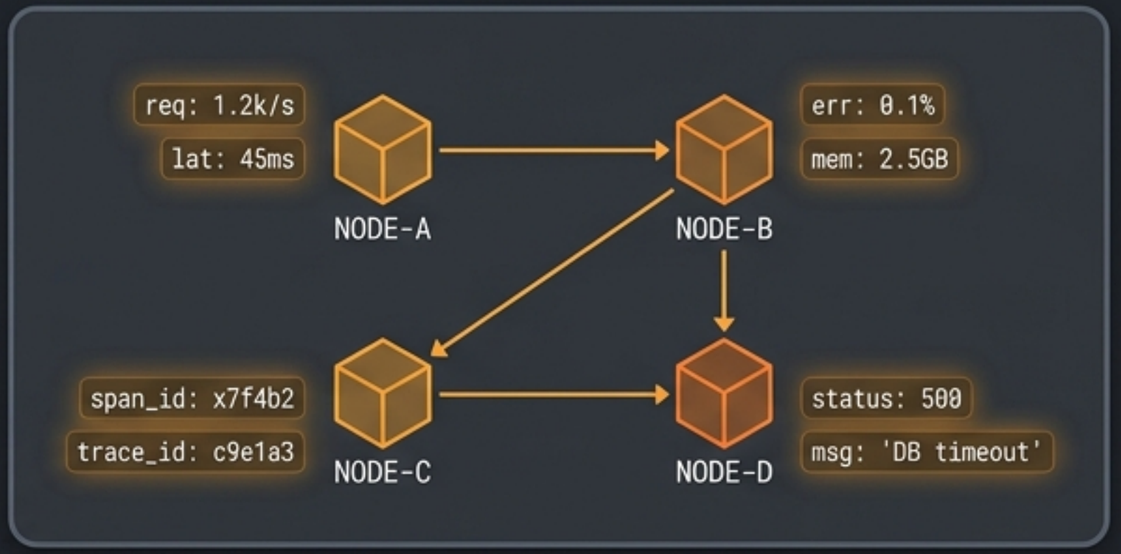


Focus: Is it broken?

Function: Tracks known metrics and alerts engineers when predefined thresholds are breached.

Example: `CPU > 90%`

OBSERVABILITY



Focus: Why is it broken?

Function: Allows engineers to ask arbitrary questions to deduce the internal state of a system from external outputs.

The DevOps Need: CI/CD pipelines and microservices fail in unpredictable, emergent ways. Teams require both tools to detect incidents quickly and debug root causes deeply.

Gemini Notebook

Introduction to Prometheus: The Engine

An open-source systems monitoring and alerting toolkit built for absolute reliability.

Time-Series Database

Stores metrics as streams of timestamped values.

Primary Use Case

Ideal for highly dynamic, containerized microservice architectures and machine-centric monitoring.



Autonomous Operation

Single-server nodes relying on local storage, completely avoiding complex distributed backends.

PromQL

A highly flexible, native query language designed for real-time dimensional data.

Gemini Notebook

Introduction to Grafana: The Interface

The open-source platform to query, visualize, alert on, and explore data—no matter where it is stored.

Data Agnostic

Natively connects to Prometheus, CloudWatch, Elasticsearch, and CI/CD tools simultaneously.

Visual Layer

Translates raw queries and time-series data into insightful, live, interactive dashboards.



Explore Mode

Built-in environment for ad-hoc data investigation without creating permanent dashboards.

Primary Use Case

The unified single pane of glass for all DevOps telemetry and infrastructure health.

Gemini Notebook

The Synergy: Storage vs. Glass

Prometheus: The Backend



- Acts as the primary data collector.
- Actively scrapes and stores metrics
- Evaluates complex mathematical PromQL expressions.

HTTP API

Grafana: The Frontend

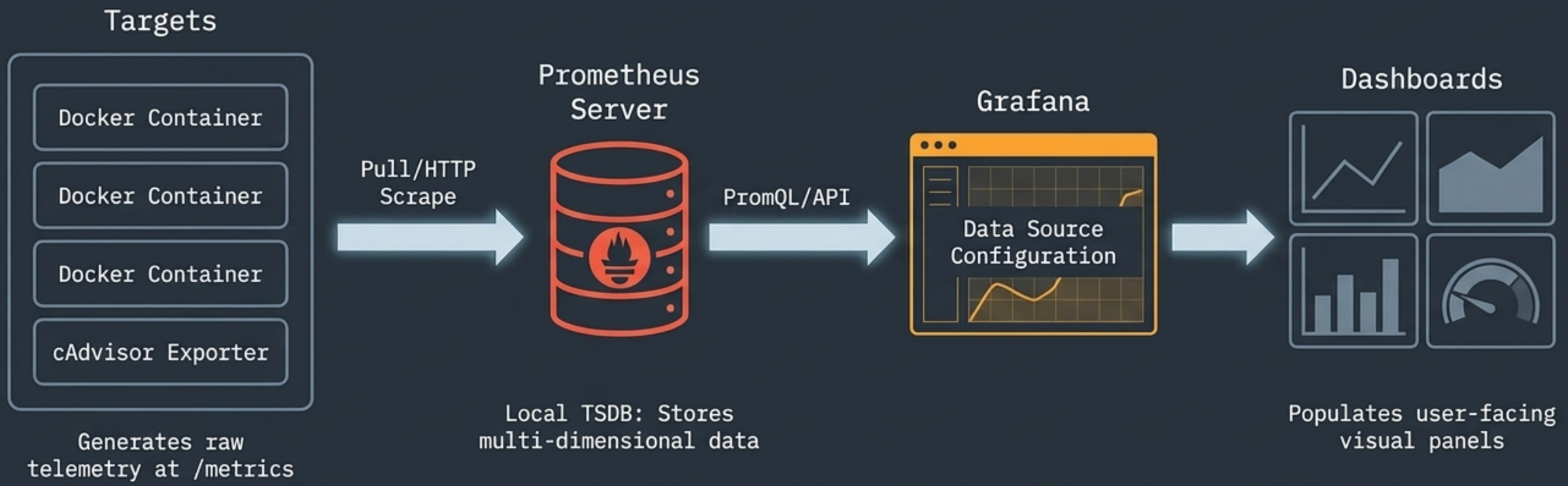


- Acts as the data visualizer.
- Fetches data dynamically via API.
- Renders human-readable charts, tables, and dashboards.

Prometheus does the heavy backend lifting of metric storage and calculation; Grafana provides the human-readable interface.

Gemini Notebook

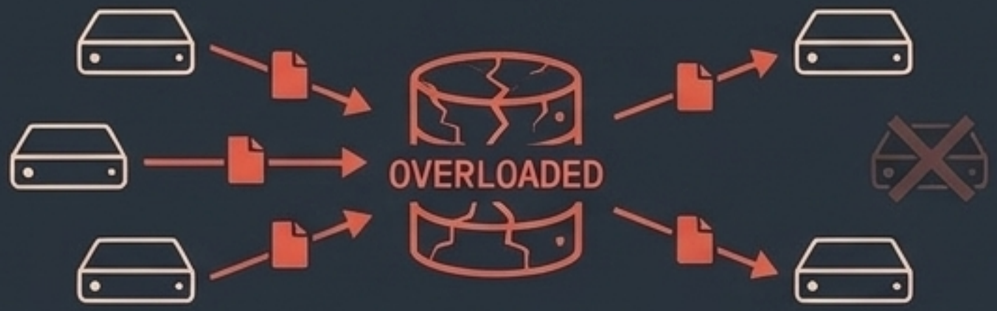
End-to-End Monitoring Architecture



Gemini Notebook

Scrape Mechanics: The Pull Model & Exporters

Push Model (Failed)



Push Model: Nodes push data to a central point. Can cause bottlenecking and undetected failures.

Pull Model (Prometheus)



Prometheus actively requests data on a scheduled interval. Eliminates server bottlenecking and instantly detects downed targets.

Exporters



Exporters: Translators for software that do not output Prometheus-formatted metrics natively.

cAdvisor: A critical exporter that exposes resource usage of running Docker containers.

Gemini Notebook

Anatomy of a Metric & PromQL

```
1
2
3
4  http_requests_total{method="GET", status="500"} 1056
5
6
7
```

Metric Name: What is being measured.

Labels: Key/Value pairs for multi-dimensional filtering.

Value: The recorded numeric data point at the current Timestamp.

>_ **Translation:** Show me the total number of HTTP requests where the method was GET and the server returned an error 500.

Gemini Notebook

The Grafana Workflow: Data to Dashboard



1. Connect Data Source

Configure Grafana to securely talk to the Prometheus HTTP API.



2. Explore & Query

Use the Explore view to interactively test PromQL queries and verify data flow.



3. Build Panels

Assign refined queries to visual components (e.g., Time-Series lines, Gauges, Stat numbers).



4. Organize Dashboards

Group panels logically to tell a cohesive, single-screen story of system health.

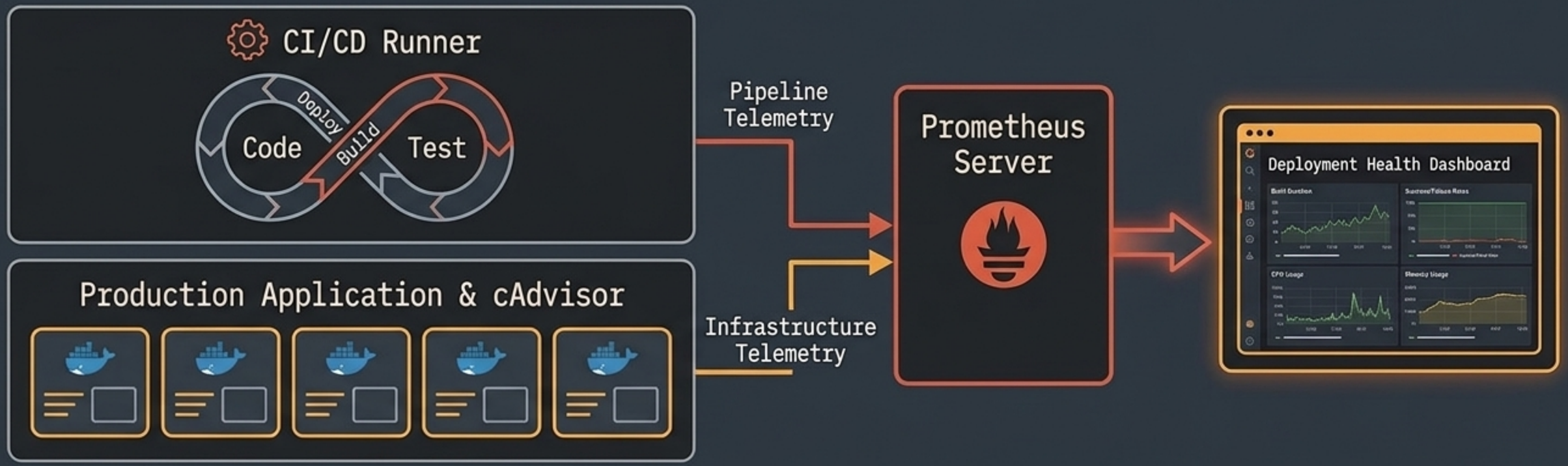
Gemini Notebook

Engineering Trade-Offs

	✅ Advantages	⚠️ Limitations
Prometheus	✅ Highly reliable standalone binary; operational during network outages. ✅ Powerful label-based querying via PromQL.	⚠️ Not designed for 100% data accuracy (e.g., per-request financial billing). ⚠️ Local storage architecture scales poorly over multi-year timeframes.
Grafana	✅ Beautiful, highly customizable user interface. ✅ Natively supports querying dozens of concurrent data sources on one screen.	⚠️ Strictly a visualizer; possesses zero internal metric storage. ⚠️ Entirely dependent on the health of underlying backends to function.

Gemini Notebook

Real-World Scenario: CI/CD & App Monitoring



- **The Scenario:** A new code commit triggers a CI/CD build, deploying to a Docker container.
- **Pipeline Metrics:** Prometheus tracks build durations and success/failure rates.
- **Container Metrics:** Prometheus scrapes cAdvisor to track CPU and memory spikes.
- **The Result:** Engineers watch a single dashboard to verify a safe deployment in real-time.

Gemini Notebook

Key Monitoring KPIs for DevOps



Service Availability (Uptime)

Is the application responding to user requests?



Pipeline Success Rate

How often do code builds pass vs. fail?



Pipeline Duration

Are build times slowing down developer velocity?



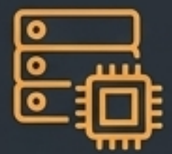
Run Rate / Throughput

How many requests is the application handling per second?



Active Builds

The current load on the CI/CD infrastructure.

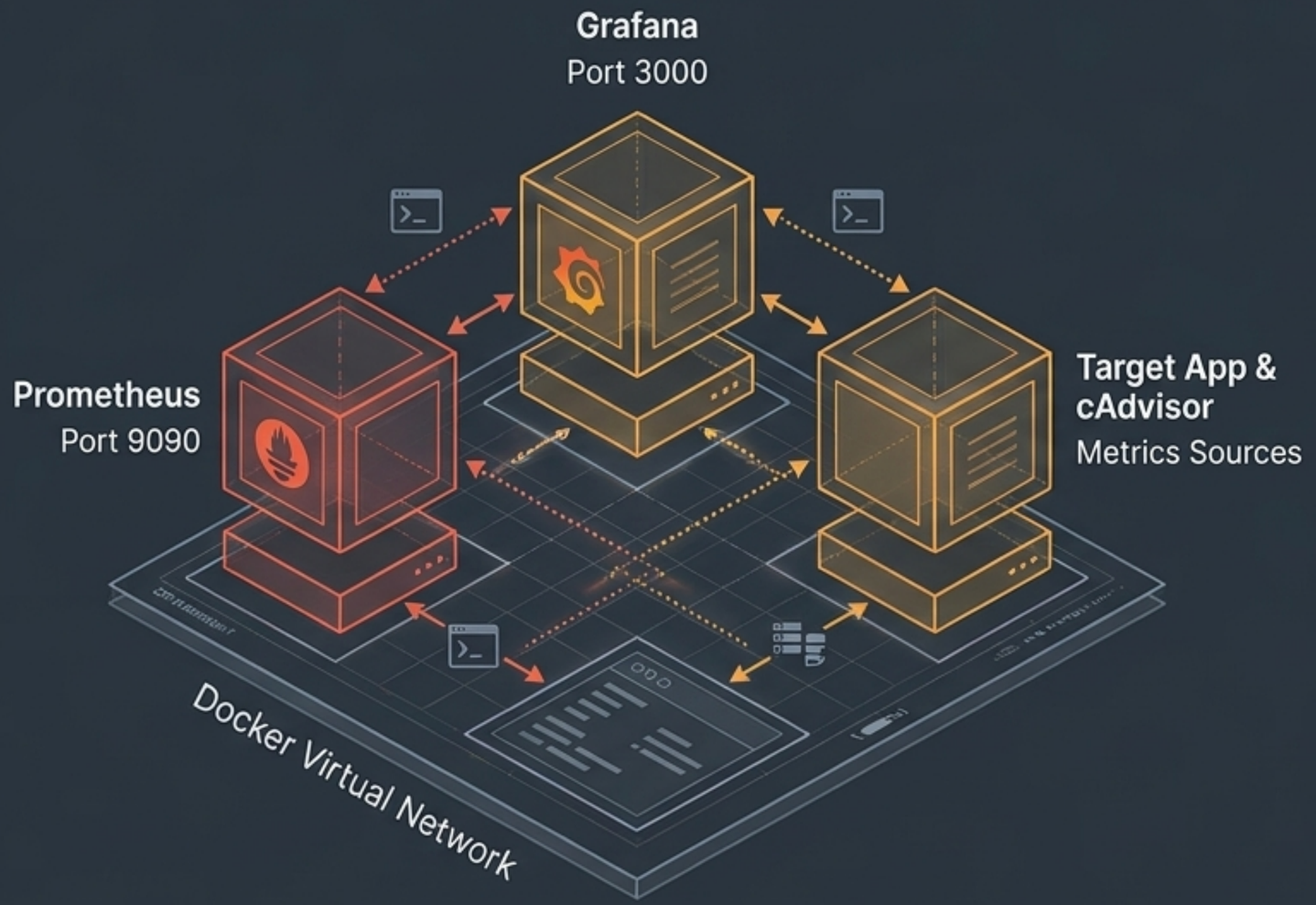


Resource Usage

CPU, Memory, and Network I/O of the Docker containers.

Gemini Notebook

Infrastructure as Code: Lab Environment Setup



Infrastructure as Code:

The entire stack is defined in a single `docker-compose.yml` file.

Internal Networking:

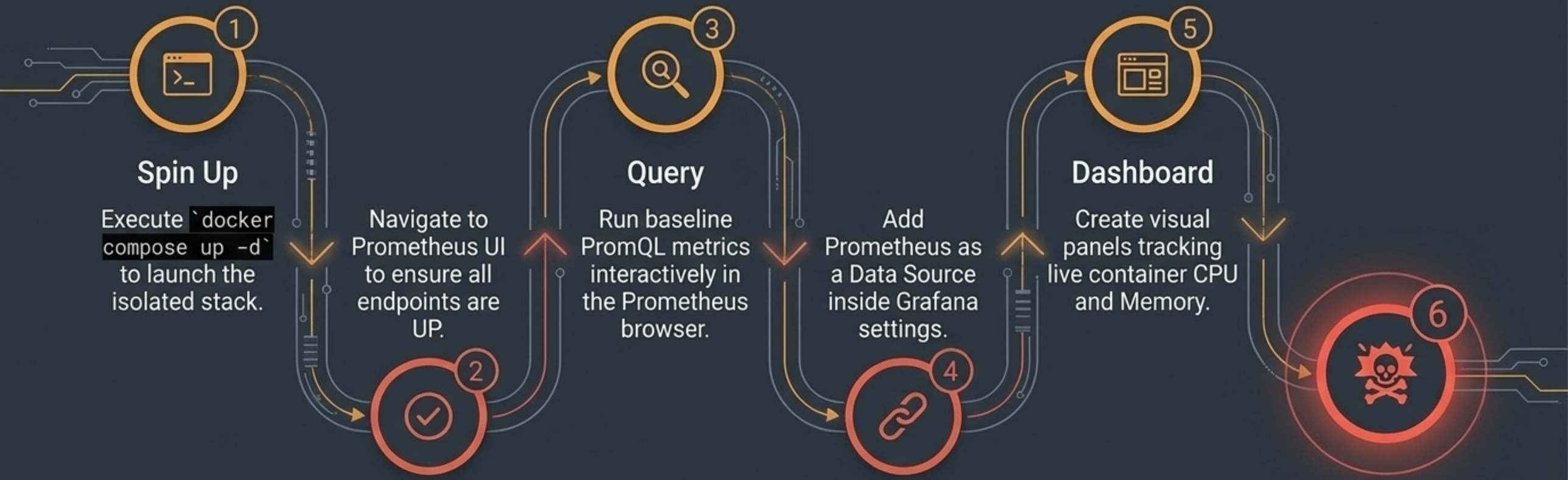
Docker local DNS allows Grafana to connect using the internal URL `http://prometheus:9090`.

Cross-Platform:

Executes identically on macOS and Windows via Docker Desktop.

Gemini Notebook

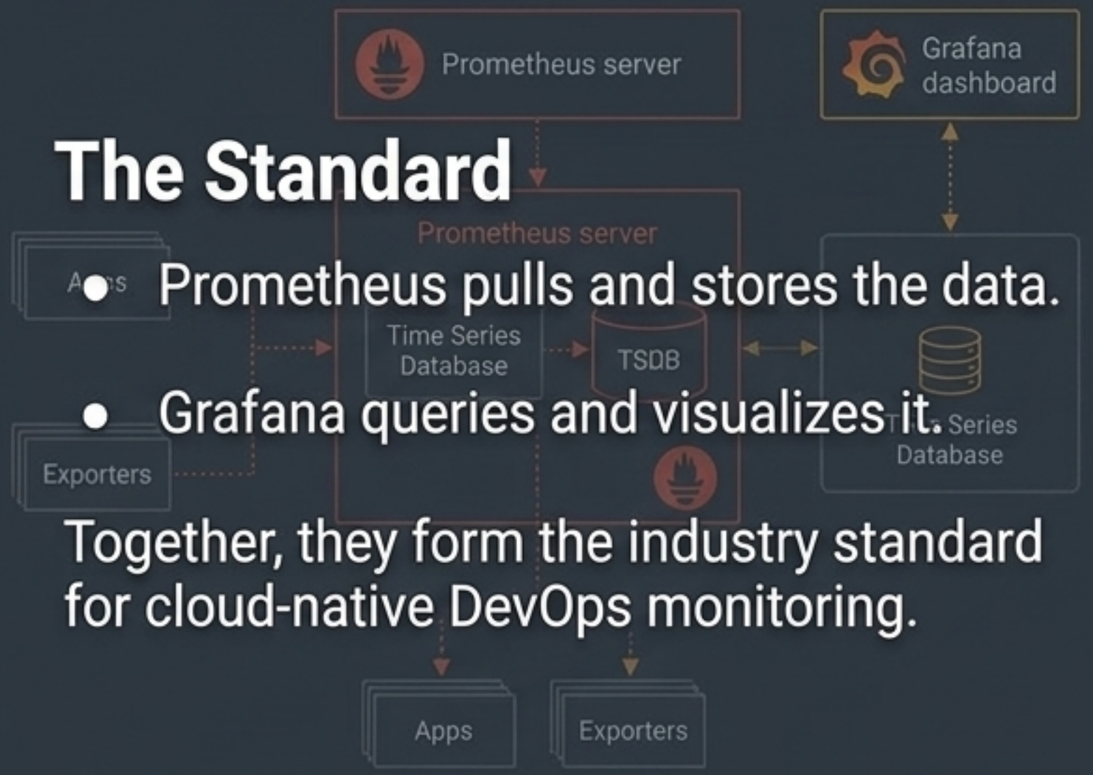
The Practical Lab Workflow



Gemini Notebook

Final Recap & Your Mission

The Concept



The Standard

- Prometheus pulls and stores the data.
- Grafana queries and visualizes it.

Together, they form the industry standard for cloud-native DevOps monitoring.

The Mission

Your Mission Today:

- [ ] Deploy the infrastructure stack via Docker.
- [ ] Write your first multi-dimensional PromQL queries.
- [ ] Build an operational single pane of glass dashboard.
- [ ] Observe and debug a real-time system failure.

```
> System ready. Proceed to command line lab.
```

Gemini Notebook